

Quiz 1

601.471/671 NLP: Self-Supervised Models

Spring 2024

Johns Hopkins University

February 2024

Complete all questions.

Use additional paper if needed.

Time: 70 minutes.

Name of student: _____

Q1. Concepts, Foundations and Big Picture

50 points

For each of the following questions, select the most appropriate answer.

1. (2 points) What best describes “self-supervised learning”?
- ☐ Models that are trained to predict outcomes but are disconnected from real-world tasks.
 - ☐ Models that are trained to predict world models, using freely-available data and are tightly connected with real-world tasks.
 - ☐ Models that use large amounts of data for machine-guided learning but have no predictive capabilities whatsoever.
 - ☐ Self-learning models that require minimal data for training.
 - ☐ Anything that talks to ChatGPT!
 - ☐ All of the above.

Solution: *Self-supervised models are statistical models that are (1) trained to be predictive models of the world, (2) using large amount of freely-available data (text, images, etc.) and (3) are tightly connected to real-world tasks that we care about. The correct answer is the 2nd one.*

2. (2 points) What best describes “language modeling”?
- ☐ Self-supervised learning over language data.
 - ☐ Learning a joint distribution over word sequence in a language.
 - ☐ Learning a marginal distribution over word sequence in a language.
 - ☐ All of the above.

Solution: *All of the above.*

3. (2 points) What is the best description of “count-based language models”?
- ☐ Models that count the number of characters in a language.
 - ☐ Models that count the number of words in a document.
 - ☐ Models that predict word sequences.
 - ☐ Models that predict word sequences based on the frequency of occurrence in the given data.
 - ☐ Models that train on a specific word count to improve computational efficiency.

Solution: *The fourth one.*

4. (2 points) What best describes the “sparsity problem” in “count-based language models”?
- ☐ The phenomenon where many possible word combinations are not present or very rare in the training data, resulting in zero or unreliable probability estimates
 - ☐ The phenomenon where the model has excess computational requirements due to large data sizes
 - ☐ The phenomenon where the model cannot handle a large vocabulary
 - ☐ All of the above.

Solution: *The first one.*

5. (2 points) What is the accurate comparison and contrast between cross-entropy and mean squared error (MSE) loss functions, and in what scenario would one be preferable to the other?
- ☐ Cross-entropy is preferable for classification problems while MSE is more suited for regression tasks.
 - ☐ Cross-entropy and MSE are both optimal for regression tasks.
 - ☐ Cross-entropy is well-suited for binary classification problems, while MSE is ideal when we have more than two classes.
 - ☐ MSE tends to penalize larger errors more heavily than the cross-entropy loss function.

Solution: *The first one.*

6. (2 points) Suppose a classifier predicts each possible class with equal probability. If there are 10 classes, what will the cross-entropy error be on a single example?
- ☐ $-\log(10)$
 - ☐ $-0.1 \log(10)$

- ☐ $-\log(0.1)$
- ☐ $-10\log(0.1)$

Solution: The third one. Cross-Entropy loss simplifies to negative log of the predicted probability for the correct class. At the start of training, we have approximately uniform probabilities for the 10 classes, or 0.1 for each class. So, that means $-\log(0.1)$ is what the loss should approximately be at the start.

7. (2 points) What is one key limitation of the Sigmoid activation function?
- ☐ It does not allow multiple outputs.
 - ☐ It can cause a 'vanishing gradients' problem during backpropagation.
 - ☐ It can only be used with linear activation functions.
 - ☐ It cannot handle multi-class classification problems.
 - ☐ None of the above.

Solution: 2nd one

8. (2 points) During Backpropagation, when the gradient flows backwards through the 'sigmoid' or 'tanh' non-linearities, it cannot change sign.
- ☐ True
 - ☐ False

Solution: True. The local gradient for both of these non-linearities is always positive.

9. (2 points) What is correct about the depth versus width trade-off when designing neural networks?
- ☐ Increasing the width of a network always guarantees more accurate results vs. increasing its depth
 - ☐ Increasing the depth of a network always guarantees more accurate results vs. increasing its width
 - ☐ Both depth and width have no impact on a network's learning capacity
 - ☐ Adding width to a neural network may require exponentially more nodes to achieve comparable capacity to networks with additional depth.
 - ☐ Adding depth to a neural network may require exponentially more nodes to achieve comparable capacity to networks with additional width.
 - ☐ Both depth and width equally influence a network's capacity to model complex patterns.

Solution: The 4th one

10. (2 points) What is an incorrect statement about backpropagation in neural networks?
- ☐ Backpropagation uses gradient descent to minimize the loss function.
 - ☐ Backpropagation adjusts the weights in the neural network based on the error of the output.
 - ☐ Backpropagation is a sequential process. Its complexity is a quadratic function of the number of layers.
 - ☐ None of the above.

Solution: The 3rd one.

11. (2 points) What is a Jacobian matrix and how it is related to neural networks?
- ☐ A matrix that represents the weights of a neural network.
 - ☐ A matrix that is used to generate random numbers for initializing weights in a neural network.
 - ☐ A matrix that is used to pre-process data before feeding it into a neural network.
 - ☐ A matrix that contains the derivatives of a function and is used to update the weights in a neural network during backprop.
 - ☐ All of the above.

Solution: A Jacobian matrix is a matrix based on the partial differentiation of a function, but first-order partial derivatives. It tells us how local perturbations the neural network input would impact the output. The correct answer is the 4th option.

12. (2 points) What is the impact of the learning rate in the process of gradient descent?
- ☐ A high learning rate may cause the algorithm to converge quickly, but risk overshooting the minimum
 - ☐ A low learning rate results in slower convergence, but risk overshooting the minimum

- ☐ The learning rate has no impact on the convergence of the gradient descent algorithm
- ☐ High learning rates completely eliminate the risk of overshooting.
- ☐ None of the above.

Solution: *The first one.*

13. (2 points) In what situation would you intentionally want to overfit a model, and why?
- ☐ When the dataset is very large and diverse, to capture every individual data point.
 - ☐ When the model is subjected to high levels of noise, to increase its robustness.
 - ☐ Never, overfitting is always undesirable.
 - ☐ When the dataset is small and noisy, you might want to overfit to reduce bias.
 - ☐ When you want the model to perfectly predict the training data, to make sure that it has enough capacity and the training works as expected.

Solution: *The last one*

14. (2 points) How does PyTorch execute the backpropagation algorithm efficiently?
- ☐ By random selection of a layer for calculation of gradient.
 - ☐ By running the forward and backward process simultaneously on multiple cores.
 - ☐ By optimally selecting the order of calculations using a computation graph.
 - ☐ By skipping the gradient calculation for some neurons.

Solution: *The 3rd one.*

15. (2 points) What happens when you call `.backward()` on a tensor in PyTorch?
- ☐ It calculates the gradient of the tensor and stores it in a separate data structure.
 - ☐ It reverses the tensor's order.
 - ☐ It requires the value of the parent node of the tensor.
 - ☐ It resets the tensor's data to zero.

Solution: *The first one.*

16. (2 points) Remember the roles of the following operations in PyTorch:

- `optimizer.step()` applies gradient updates on parameters of a neural network.
- `optimizer.zero_grad()` resets the gradient values (sets them to None).

You're writing a PyTorch training loop. What is the correct order of calling PyTorch functions for backpropagation process?

- ☐ `loss.backward()` → `optimizer.step()` → `optimizer.zero_grad()`
- ☐ `optimizer.step()` → `loss.backward()` → `optimizer.zero_grad()`
- ☐ `loss.backward()` → `optimizer.zero_grad()` → `optimizer.step()`
- ☐ None of the above.
- ☐ All of the above.

Solution: *The first one.*

17. (2 points) What typically causes vanishing or exploding gradients in a neural network?
- ☐ Not using appropriate activation functions, such as ReLU.
 - ☐ The deep architecture of the network and the use of certain activation functions like sigmoid or tanh.
 - ☐ Specific weights and bias strategies, like zero initialization.
 - ☐ Overfitting or underfitting on the provided training data.
 - ☐ None of the above

Solution: *2nd choice*

18. (2 points) What is the importance of data normalization when training a neural network?
- ☐ It ensures all input features contribute equally to the model's prediction, improving training efficiency and convergence speed.
 - ☐ It increases the range of input data, making the network less sensitive to small changes.

- ☐ It enhances the model's ability to find more complex patterns without requiring normalization.
- ☐ It reduces the size of the input data, leading to less computational power needed for training.

Solution: *The first one.*

19. (2 points) How does the Dropout prevent overfitting in neural networks?
- ☐ By randomly setting activations to linear functions during training
 - ☐ By randomly selecting neurons to be dropped from the model during training
 - ☐ By reducing the learning rate
 - ☐ By increasing the size of the dataset
 - ☐ None of the above.

Solution: *2nd one.*

20. (2 points) What correctly describes dropout layer in deep nets during train and test times?
- ☐ During training, dropout randomly omits a subset of features; during testing, it omits the same subset of features to ensure consistency.
 - ☐ Dropout is applied during training to randomly deactivate a set of neurons to prevent overfitting, and during testing, it is not used, allowing all neurons to contribute to the output.
 - ☐ During training, dropout increases the learning rate selectively for certain neurons, and during testing, it equalizes the learning rate across all neurons.
 - ☐ Dropout layers are only used during testing to simulate a diverse set of neural network architectures by randomly deactivating neurons.
 - ☐ None of the above.

Solution: *2nd one*

21. (2 points) What is the primary purpose of the `torch.nn.Module` class in PyTorch?
- ☐ To handle the loading and saving of models
 - ☐ To provide mathematical operations for tensors
 - ☐ To optimize model parameters
 - ☐ To provide a base for all neural network modules.

Solution: *The last one*

22. (2 points) What is a reasonable way for initializing neural network weights in PyTorch?
- ☐ Setting all weights to zero
 - ☐ Setting all weights to a non-zero constant
 - ☐ Random initialization using normal distribution with positive mean.
 - ☐ Random initialization using normal distribution with zero mean.

Solution: *The fourth one*

23. (2 points) What issue can arise when using non-zero-centered data during backpropagation?
- ☐ The learning rate automatically adjusts to compensate, leading to unpredictable training behavior.
 - ☐ Gradients in certain directions will be impossible, leading to inefficient training.
 - ☐ Non-zero-centered data enables faster training enabled with a more efficient representations of weights.
 - ☐ All gradients become zero, making it impossible for the network to learn from the data.

Solution: 2nd one

24. (4 points) Suppose that you are training a neural network for sentence classification, but you notice that the training loss is much lower than the validation loss. Which of the following can be used to address the issue? Select all the options that apply.
- ☐ Use a network with fewer layers
 - ☐ Decrease dropout probability
 - ☐ Reduce the training data
 - ☐ Reduce the width
 - ☐ Increase the size of each hidden layer

Solution: 1, 4 – the model is now overfitting, and all of these are valid techniques to address it. However, since dropout is a form of regularization, then decreasing it (B) will have the opposite effect, as will increasing network size (D).

Q2. Language Modeling and N-Gram Models

20 points

1. (4 points) Recall the definition of perplexity:

$$\text{ppl}(w_1, \dots, w_n) = 2^H \text{ where } H = -\frac{1}{n} \sum_{i=1}^n \log_2 p(w_i | w_1, \dots, w_{i-1})$$

Assume that perplexity is measured over entirety of a large natural language corpus with n words (for a large n). Also assume that this corpus is made of V -many unique words.

(a) If the model always knows what word should follow a given context, what is the perplexity of the model? (b) Under what conditions the model achieves a perplexity of V ? Show your argument.

Solution: Answer: (a) 1, (b) maximum is the number of the words in the language and when get that when our LM predicts completely randomly.

2. (5 points) Consider the following vocabulary, $V = \{\text{BOS}, \text{EOS}, \text{who}, \text{framed}, \text{jiefu}, \text{class}, \text{the}\}$ where BOS is the dummy token indicating the beginning of a sentence, and EOS indicates the end of a sentence. Consider the following training data:

```
BOS who EOS
BOS framed EOS
BOS who framed jiefu who EOS
BOS jiefu class EOS
BOS who framed jiefu class EOS
BOS jiefu framed who EOS
BOS who framed the class EOS
BOS jiefu framed the class EOS
```

Compute the following probabilities. Show your work. Hint: The combinations of bigrams of interest (including BOS) are given to you in the table. Feel free to write your response in the table or the box below that.

n -gram	Probability
$P(\text{jie fu} \text{BOS})$	
$P(\text{framed} \text{jie fu})$	
$P(\text{who} \text{framed})$	
$P(\text{EOS} \text{who})$	

Solution: First we count unigram counts: BOS: 8, jie fu: 5, framed: 6, who: 6
Then we count bigram counts: BOS jie fu: 3, jie fu framed: 2, framed who: 1, who EOS: 3
This gives us the following table:

n -gram	Probability
$P(\text{jie fu} \text{BOS})$	3/8
$P(\text{framed} \text{jie fu})$	2/5
$P(\text{who} \text{framed})$	1/6
$P(\text{EOS} \text{who})$	3/6

3. (6 points) Using the above estimate, provide the probabilities for $s_1 := \text{"BOS jie fu framed who EOS"}$. No need to simplify the derivations. Compute the probability using the bigram probabilities that you computed in Q2.2. Hint: $P(s_1) = P(\text{BOS}) \times P(\text{jie fu}|\text{BOS}) \times \dots$ and since all sentences start with $P(\text{BOS})$ you can ignore this term.

Solution:

$$P(s_1) = P(\text{BOS})P(\text{jie fu}|\text{BOS})P(\text{framed}|\text{jie fu})P(\text{who}|\text{framed})P(\text{EOS}|\text{who}) = 1 \times 3/8 \times 2/5 \times 1/6 \times 3/6 = 0.0125$$

4. (5 points) Compute the perplexity of the previous sentence. Remember that BOS should not be included when calculating the length of the sentence (only the bigrams). No need to simplify the derivations.

Solution:

$$H = -\frac{1}{4}[\log_2 3/8 + \log_2 2/5 + \log_2 1/6 + \log_2 3/6] = 1.33$$

$$\text{ppl}(w_1, \dots, w_n) = 2^H = 2.51$$

Q3. Neural Nets and Backpropagation

30 points

1. (5 points) Compute the Jacobian matrix of the given function $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$

$$f(\theta = [x, y, z]) = [xyz, x^2, y/z]$$

where $x, y, z \in \mathbb{R}$.

Solution:

$$J_f([x, y, z]) = \begin{bmatrix} yz, xz, xy \\ 2x, 0, 0 \\ 0, 1/z, -y/z^2 \end{bmatrix}$$

2. (5 points) In class, we learned how the ReLU activation function ($\text{ReLU}(z) = \max(0, z)$) could “die” or become saturated/inactive when the input is negative. A friend of yours suggests the use of another activation function $f(z) = \max(0.2z, 2z)$ which he claims will remove the saturation problem. Would this address the problem? Why or why not? (less than 3 sentences)

Solution: Yes, – it addresses the saturation problem of ReLU and also is not a purely linear function. In fact, this is just an instance of the Leaky ReLU non-linearity, frequently used to train neural nets in practice.

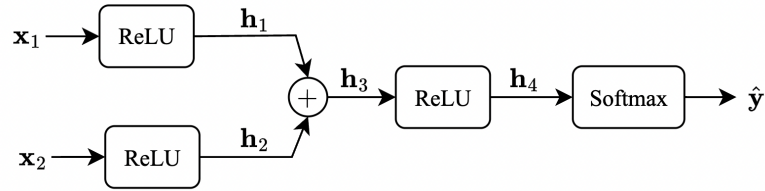
3. (5 points) Compute the gradient of the ReLU function, i.e. derive $\frac{\partial}{\partial z} \text{ReLU}(z)$.

Hint: You can use the following notation: $1\{x > 0\} = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

Solution:

$$\frac{\partial}{\partial z} \text{ReLU}(z) = 1\{z > 0\}$$

4. (15 points) In the following neural network, compute the gradient $\frac{\partial J}{\partial \mathbf{x}_2}$.



$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x}_1, \mathbf{z}_2 = \mathbf{W}_2 \mathbf{x}_2, \mathbf{h}_1 = \text{ReLU}(\mathbf{z}_1), \mathbf{h}_2 = \text{ReLU}(\mathbf{z}_2), \mathbf{h}_3 = \mathbf{h}_1 + \mathbf{h}_2, \mathbf{z}_3 = \mathbf{W}_3 \mathbf{h}_3, \mathbf{h}_4 = \text{ReLU}(\mathbf{z}_3),$$

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{h}_4), \mathbf{J} = \text{CE}(\mathbf{y}, \hat{\mathbf{y}}), \mathbf{x} \in \mathbb{R}^d, \mathbf{W}_{1,2,3} \in \mathbb{R}^{d \times d}, \hat{\mathbf{y}} \in \mathbb{R}^d$$

You are allowed (and highly encouraged) to use variables to represent intermediate gradients.

Hint 1: Compute these gradients in order to build up your answer: $\frac{\partial J}{\partial \mathbf{h}_4}, \frac{\partial J}{\partial \mathbf{h}_3}, \frac{\partial J}{\partial \mathbf{z}_3}, \frac{\partial J}{\partial \mathbf{h}_2}, \frac{\partial J}{\partial \mathbf{z}_2}, \frac{\partial J}{\partial \mathbf{x}_2}$. Show your work so we are able to give partial credit!

Hint 2: Recall that *downstream* = *upstream* * *local*.

Hint 3: Assume that $\frac{\partial J}{\partial \mathbf{h}_4} = \hat{\mathbf{y}} - \mathbf{y}$. You will prove this in HW4!

Solution: From hint3, we know:

$$\delta_1 = \frac{\partial J}{\partial \mathbf{h}_4} = \hat{\mathbf{h}}_4 - \mathbf{y}$$

$$\delta_2 = \frac{\partial J}{\partial \mathbf{z}_3} = \frac{\partial J}{\partial \mathbf{h}_4} \frac{\partial \mathbf{h}_4}{\partial \mathbf{z}_3} = \delta_1 1\{\mathbf{z}_3 > 0\}$$

$$\delta_3 = \frac{\partial J}{\partial \mathbf{h}_3} = \frac{\partial J}{\partial \mathbf{z}_3} \frac{\partial \mathbf{z}_3}{\partial \mathbf{h}_3} = \delta_2 \mathbf{W}_3$$

$$\frac{\partial J}{\partial \mathbf{h}_2} = \frac{\partial J}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} = \delta_3$$

$$\delta_4 = \frac{\partial J}{\partial \mathbf{z}_2} = \frac{\partial J}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{z}_2} = \delta_3 1\{\mathbf{z}_2 > 0\}$$

$$\frac{\partial J}{\partial \mathbf{x}_2} = \frac{\partial J}{\partial \mathbf{z}_2} \frac{\partial \mathbf{z}_2}{\partial \mathbf{x}_2} = \delta_4 \mathbf{W}_2$$